

Function Specification Document

MAFGA Multi-Agent POC

Version: 1.0.0

Date: April 2026

Status: Implementation Complete

Table of Contents

1. Overview
 2. API Endpoints
 3. Agent Functions
 4. Orchestration Functions
 5. MCP Server Functions
 6. A2A Protocol Functions
 7. Middleware Functions
 8. Memory Functions
 9. Authentication Functions
 10. Observability Functions
-

1. Overview

This document specifies all public functions and interfaces in the MAFGA Multi-Agent POC system. Functions are organized by module with detailed signatures, parameters, return types, and behavior descriptions.

2. API Endpoints

2.1 Health Endpoints

GET /health

Description: Returns system health status and dependency checks.

Response:

```
{
  "status": "healthy",
  "version": "1.0.0",
  "uptime_seconds": 3600.5,
  "services": [
    {"name": "redis", "status": "healthy", "latency_ms": 1.2}
  ]
}
```

GET /ready

Description: Kubernetes readiness probe.

Response: {"ready": true}

GET /live

Description: Kubernetes liveness probe.

Response: {"alive": true}

2.2 Agent Endpoints

GET /agents

Description: List all available agents.

Response:

```
{
  "agents": [
    {
      "name": "MerchPlanner",
      "description": "Merchandise planning agent",
      "capabilities": ["inventory_analysis"],
      "available_tools": ["query_sales"]
    }
  ],
  "count": 6
}
```

GET /agents/{agent_name}

Description: Get details for a specific agent.

Parameters:

Name	Type	Required	Description
agent_name	string	Yes	Agent identifier

Response: AgentInfo object

POST /agents/invoke

Description: Invoke an agent with a query.

Request Body:

```
{
  "agent_name": "MerchPlanner",
  "query": "Analyze paint category sales",
}
```

```
"context": {"region": "West"},
"conversation_id": "conv-123"
}
```

Response:

```
{
  "status": "success",
  "agent_name": "MerchPlanner",
  "content": "Analysis results...",
  "conversation_id": "conv-123",
  "tool_calls": [],
  "metadata": {"tokens_used": 150}
}
```

2.3 Orchestration Endpoints

POST /orchestration/run

Description: Execute multi-agent orchestration.

Request Body:

```
{
  "goal": "Analyze category and recommend strategy",
  "orchestration_type": "magentic",
  "require_approval": true,
  "context": {}
}
```

Response:

```
{
  "status": "completed",
  "result": "Synthesized analysis...",
  "plan": {
    "plan_id": "plan-abc123",
    "goal": "Analyze category",
    "steps": [],
    "approved": true
  }
}
```

GET /orchestration/approvals

Description: List pending approval requests.

Response:

```
{
  "approvals": [],
  "count": 0
}
```

POST /orchestration/approvals/{request_id}

Description: Approve or reject a pending request.

Request Body:

```
{
  "request_id": "req-123",
  "action": "approve",
  "comments": "Looks good"
}
```

3. Agent Functions

3.1 BaseRetailAgent

```
class BaseRetailAgent(ABC):
    """Abstract base class for all retail domain agents."""
```

```
__init__(name, description, client, mcp_servers, skills)
```

Description: Initialize an agent instance.

Parameter	Type	Required	Description
name	str	Yes	Unique agent identifier
description	str	Yes	Human-readable description
client	ChatClient	Yes	LLM chat client
mcp_servers	list[BaseMCPServer]	No	Available MCP tools
skills	list[Skill]	No	Agent skills

```
async invoke(query, context) -> AgentResponse
```

Description: Process a query and return a response.

Parameter	Type	Required	Description
query	str	Yes	User query/task
context	dict	No	Additional context

Returns: AgentResponse with content, tool_calls, metadata

```
get_tools() -> list[MCPTool]
```

Description: Get all available tools from MCP servers.

Returns: List of tool definitions

```
to_dict() -> dict
```

Description: Serialize agent to dictionary.

3.2 AgentFactory

`create_chat_client(provider) -> ChatClient`

Description: Create a cloud-agnostic chat client.

Parameter	Type	Required	Description
provider	str	No	Provider name (default from env)

Supported Providers:

- `azure_foundry` - Azure AI Foundry
- `azure_openai` - Azure OpenAI Service
- `openai` - OpenAI API
- `anthropic` - Anthropic Claude
- `gemini` - Google Gemini
- `ollama` - Local Ollama
- `bedrock` - AWS Bedrock

Returns: Configured `ChatClient` instance

3.3 Domain Agent Functions

MerchPlannerAgent

`async def invoke(query: str, context: dict = None) -> AgentResponse`

Capabilities: `inventory_analysis`, `assortment_planning`, `sales_forecasting`

MCP Tools: `query_sales_data`, `get_inventory_levels`, `get_category_performance`

SpacePlannerAgent

`async def invoke(query: str, context: dict = None) -> AgentResponse`

Capabilities: `planogram_design`, `space_allocation`, `fixture_planning`

MCP Tools: `query_space_data`, `get_planogram_data`, `get_fixture_info`

LoyaltyAgent

`async def invoke(query: str, context: dict = None) -> AgentResponse`

Capabilities: `loyalty_analysis`, `personalization`, `offer_targeting`

MCP Tools: `get_customer_profile`, `get_preferences`, `get_regional_settings`

ProductsFinderAgent

`async def invoke(query: str, context: dict = None) -> AgentResponse`

Capabilities: product_search, recommendations, availability_check
MCP Tools: search_products, check_availability, get_product_details

CommercialSalesAgent

```
async def invoke(query: str, context: dict = None) -> AgentResponse
```

Capabilities: lead_management, opportunity_tracking, account_analysis
MCP Tools: search_opportunities, get_contacts, create_task

CampaignAnalystAgent

```
async def invoke(query: str, context: dict = None) -> AgentResponse
```

Capabilities: campaign_analysis, weather_impact, promotional_planning
MCP Tools: get_weather_forecast, get_historical_weather, query_campaign_data

4. Orchestration Functions

4.1 MagenticOrchestrator

```
__init__(client, agents, human_approval_callback, max_rounds, checkpoint_store)
```

Description: Initialize the Magentic orchestrator.

Parameter	Type	Required	Description
client	ChatClient	Yes	LLM for planning
agents	dict[str, BaseRetailAgent]	Yes	Agent registry
human_approval_callback	Callable	No	HITL callback
max_rounds	int	No	Max execution rounds (default: 20)
checkpoint_store	CheckpointStore	No	State persistence

```
async run(goal, context, require_approval) -> dict
```

Description: Execute orchestration for a goal.

Parameter	Type	Required	Description
goal	str	Yes	User's goal
context	dict	No	Additional context
require_approval	bool	No	Require HITL approval

Returns:

```
{  
    "result": str, # Synthesized result
```

```

    "plan": dict,                # Execution plan
    "agent_responses": dict,     # Individual agent results
    "status": str                # "completed" | "rejected"
}

```

async _create_plan(goal) -> MagenticPlan

Description: Use LLM to create execution plan.

async _execute_plan(plan) -> dict[str, AgentResponse]

Description: Execute plan steps respecting dependencies.

async _synthesize_results(plan, agent_responses) -> str

Description: Combine agent results into coherent response.

4.2 TrafficRouter

__init__(client, agents, strategy, default_agent)

Description: Initialize traffic router.

Parameter	Type	Required	Description
client	ChatClient	Yes	LLM for classification
agents	dict	Yes	Agent registry
strategy	RoutingStrategy	No	Routing strategy
default_agent	str	No	Fallback agent

async route(task, context) -> dict

Description: Route task to appropriate agent.

Returns:

```

{
    "result": str,
    "routing": {
        "agent_name": str,
        "confidence": float,
        "reasoning": str
    },
    "status": str
}

```

set_strategy(strategy) -> None

Description: Change routing strategy at runtime.

Strategies:

- INTENT_BASED - LLM intent classification
 - ROUND_ROBIN - Simple rotation
 - CAPABILITY_MATCH - Keyword matching
-

4.3 ParallelExecutor

`__init__(agents, timeout_seconds, aggregation)`

Description: Initialize parallel executor.

Parameter	Type	Required	Description
agents	dict	Yes	Agent registry
timeout_seconds	float	No	Per-task timeout
aggregation	AggregationStrategy	No	Result aggregation

`async run(tasks) -> dict`

Description: Execute tasks in parallel.

Parameter	Type	Required	Description
tasks	list[ParallelTask]	Yes	Tasks to execute

Returns:

```
{
    "results": list[dict],
    "summary": {
        "total_tasks": int,
        "completed": int,
        "failed": int,
        "total_duration_ms": float
    },
    "status": str
}
```

`async broadcast(query, agent_names, context) -> dict`

Description: Send same query to multiple agents.

4.4 HumanApprovalManager

`async request_approval(title, description, plan_data, timeout_minutes) -> ApprovalRequest`

Description: Create approval request and wait.

`approve(request_id, approver, comments) -> bool`

Description: Approve a pending request.

`reject(request_id, approver, reason) -> bool`

Description: Reject a pending request.

`get_pending() -> list[ApprovalRequest]`

Description: Get all pending approvals.

`create_auto_approve_callback() -> Callable`

Description: Create callback that auto-approves all plans.

5. MCP Server Functions

5.1 BaseMCPServer

```
class BaseMCPServer(ABC):  
    """Abstract base for MCP tool servers."""
```

`list_tools() -> list[MCPTool]`

Description: Return available tool definitions (tools/list).

`async call_tool(tool_call) -> MCPToolResult`

Description: Execute a tool call (tools/call).

Parameter	Type	Required	Description
tool_call	MCPToolCall	Yes	Tool name and arguments

5.2 SnowflakeMCP Tools

Tool	Description	Parameters
query_sales_data	Query sales data	category, date_range, region
get_inventory_levels	Get inventory by SKU	sku, location
get_category_performance	Category metrics	category, metric_type

5.3 SalesforceMCP Tools

Tool	Description	Parameters
search_opportunities	Search CRM opportunities	status, owner, min_amount

get_contacts	Get account contacts	account_id
create_task	Create CRM task	subject, due_date, related_to

5.4 WeatherMCP Tools

Tool	Description	Parameters
get_weather_forecast	Get forecast	location, days
get_historical_weather	Historical data	location, date_range

5.5 ItemsAPIMCP Tools

Tool	Description	Parameters
search_products	Search catalog	query, category, filters
check_availability	Check stock	sku, location
get_product_details	Product info	sku

5.6 PersonalisationMCP Tools

Tool	Description	Parameters
get_customer_profile	Profile data	customer_id
get_preferences	Preferences	customer_id
get_recommendations	Personalized recs	customer_id, context

5.7 LocalisationMCP Tools

Tool	Description	Parameters
get_regional_settings	Regional config	region_code
get_store_info	Store details	store_id
translate_content	Localization	content, target_locale

6. A2A Protocol Functions

6.1 A2AClient

async send(recipient, content, conversation_id, context) -> A2AResponse

Description: Send message to external agent.

Parameter	Type	Required	Description
recipient	str	Yes	Target agent name
content	str	Yes	Message content

conversation_id	str	No	Conversation tracking
context	dict	No	Additional context

async handoff(recipient, content, conversation_id, full_context) -> A2AResponse

Description: Hand off conversation to another agent.

add_endpoint(agent_name, endpoint) -> None

Description: Register new agent endpoint.

get_conversation(conversation_id) -> list[A2AMessage]

Description: Get conversation history.

6.2 A2AServer

register_handler(handler) -> None

Description: Register message handler.

async process_message(message_data) -> dict

Description: Process incoming A2A message.

create_fastapi_router() -> APIRouter

Description: Create FastAPI router for A2A endpoint.

7. Middleware Functions

7.1 MiddlewareBase

```
class MiddlewareBase(ABC):  
    """Base class for middleware components."""
```

async process(request, context, next_middleware) -> MiddlewareResult

Description: Process request through middleware chain.

7.2 RBACMiddleware

async process(request, context, next_middleware) -> MiddlewareResult

Description: Check role-based access control.

Access Rules:

```
AGENT_ROLES = {  
    "MerchPlanner": ["analyst", "planner", "admin"],  
    "SpacePlanner": ["planner", "admin"],  
    "CommercialSales": ["sales", "admin"],  
}
```

7.3 ContentSafetyMiddleware

async process(request, context, next_middleware) -> MiddlewareResult

Description: Filter unsafe content.

Checks:

- PII detection (email, phone, SSN patterns)
 - Blocked keyword filtering
 - Content length limits
-

7.4 AuditLogMiddleware

async process(request, context, next_middleware) -> MiddlewareResult

Description: Log all requests for compliance.

Log Fields:

```
{  
    "timestamp": datetime,  
    "request_id": str,  
    "user_id": str,  
    "agent_name": str,  
    "action": str,  
    "status": str,  
    "duration_ms": float  
}
```

8. Memory Functions

8.1 MemoryProvider

```
class MemoryProvider(ABC):  
    """Abstract memory provider interface."""
```

```
async store(conversation_id, message) -> None
```

Description: Store conversation message.

```
async get_history(conversation_id, limit) -> list[dict]
```

Description: Retrieve conversation history.

```
async clear(conversation_id) -> None
```

Description: Clear conversation memory.

8.2 RedisMemoryProvider

Connection: Uses Redis sorted sets for ordering.

```
async store(conversation_id, message) -> None
```

Description: Store with timestamp score.

```
async get_history(conversation_id, limit) -> list[dict]
```

Description: Retrieve ordered by timestamp.

8.3 CheckpointStore

```
async save(checkpoint_id, data) -> None
```

Description: Save checkpoint state.

```
async load(checkpoint_id) -> CheckpointData
```

Description: Load checkpoint state.

```
async list_checkpoints() -> list[str]
```

Description: List all checkpoints.

9. Authentication Functions

9.1 EntraIDAuth

```
async validate_token(token) -> UserContext
```

Description: Validate JWT token against EntraID.

Returns:

```
@dataclass
class UserContext:
    user_id: str
    email: str
    name: str
    roles: list[str]
    claims: dict
```

get_auth_url() -> str

Description: Get OAuth authorization URL.

async exchange_code(code) -> TokenResponse

Description: Exchange auth code for tokens.

10. Observability Functions

10.1 TelemetryProvider

initialize() -> None

Description: Initialize OpenTelemetry providers.

get_tracer(name) -> Tracer

Description: Get tracer instance.

get_meter(name) -> Meter

Description: Get meter instance.

shutdown() -> None

Description: Graceful shutdown.

10.2 Tracing Decorators

@trace_agent_call(agent_name, operation)

Description: Trace agent invocations.

@trace_tool_call(tool_name, server_name)

Description: Trace MCP tool calls.

@trace_orchestration(orchestration_type)

Description: Trace orchestration operations.

10.3 AgentMetrics

record_request(agent_name, attributes) -> None

Description: Record new request.

record_success(agent_name, attributes) -> None

Description: Record successful request.

record_error(agent_name, error_type, attributes) -> None

Description: Record failed request.

record_latency(agent_name, latency_ms, attributes) -> None

Description: Record request latency.

measure_latency(agent_name, attributes) -> ContextManager

Description: Context manager for latency measurement.

Appendix: Data Types

AgentResponse

```
@dataclass
class AgentResponse:
    content: str
    agent_name: str
    tool_calls: list[dict] = []
    metadata: dict = {}
    timestamp: datetime
```

MCPTool

```
@dataclass
class MCPTool:
    name: str
    description: str
```

```
input_schema: dict
```

MCPToolCall

```
@dataclass
class MCPToolCall:
    name: str
    arguments: dict
```

MCPToolResult

```
@dataclass
class MCPToolResult:
    tool_name: str
    result: Any
    is_error: bool = False
    error_message: str = None
```

MagenticPlan

```
@dataclass
class MagenticPlan:
    plan_id: str
    goal: str
    steps: list[PlanStep]
    created_at: datetime
    approved: bool
```

PlanStep

```
@dataclass
class PlanStep:
    step_id: str
    agent_name: str
    task: str
    dependencies: list[str]
    status: PlanStepStatus
    result: str
```